

Distributed Systems: Sharding and Transactions

CISS 2026WS - Exam-focused course notes

1. Sharding fundamentals

Sharding partitions a dataset so different nodes own different subsets. A shard key maps each record to a shard and strongly influences load balance and query efficiency.

Range partitioning supports range scans but can create hot shards for sequential keys. Hash partitioning spreads keys more evenly but scatters range queries.

Resharding moves ownership and data while requests continue. Consistent hashing reduces the fraction of keys moved when nodes join or leave.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

2. Distributed transaction goals

A distributed transaction spans multiple resource managers but should preserve atomicity: all participants commit or all abort.

Atomic commit is different from consensus. Atomic commit decides commit only if every participant can commit; consensus chooses one proposed value despite failures.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

3. Two-phase commit: prepare

In phase one, the coordinator sends PREPARE. Each participant validates the transaction, forces required state to durable storage, acquires or retains necessary locks, and votes YES or NO.

A YES vote is a promise that the participant can commit later even after recovery. A NO vote allows the coordinator to decide ABORT immediately.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

4. Two-phase commit: decision and blocking

In phase two, the coordinator records the global decision and sends COMMIT if every vote was YES; otherwise it sends ABORT. Participants durably record and acknowledge the decision.

Two-phase commit can block when the coordinator fails after participants vote YES but before they learn the decision. Prepared participants cannot safely decide alone because either global decision may already exist.

Coordinator replication improves availability but does not change the core atomic-commit rule.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

5. Three-phase commit and timing assumptions

Three-phase commit adds a pre-commit phase to reduce uncertainty, but non-blocking behavior depends on bounded-delay and failure-detection assumptions that do not hold in a fully asynchronous network with partitions.

Therefore 3PC is not a universal replacement for 2PC in practical partitionable systems.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

6. Sagas

A saga decomposes a long transaction into local transactions. If a later step fails, compensating actions semantically undo earlier completed steps.

Compensation is not necessarily a physical rollback. Refunding a payment is a new business operation and may remain visible in audit history.

Sagas improve autonomy and availability but expose intermediate states and require explicit handling of retries, idempotency, and compensation failures.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

7. Isolation and anomalies

Dirty reads observe uncommitted data. Non-repeatable reads observe a changed committed value within one transaction. Phantoms occur when repeating a predicate query returns a different set of rows.

Write skew occurs when concurrent transactions read overlapping data and write different items, jointly violating an invariant. Snapshot isolation prevents many anomalies but can allow write skew.

Serializable execution is equivalent to some serial order and prevents write skew when the invariant is correctly represented.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

8. MVCC and an exam-oriented comparison

Multi-version concurrency control stores multiple versions so readers can use a consistent snapshot without blocking writers. Garbage collection eventually removes versions no active snapshot can require.

Use 2PC when atomic commitment across participants is mandatory and blocking risk is accepted. Use a saga when business-level compensation and temporary intermediate states are acceptable.

Sharding decides data placement; replication decides how copies are maintained; transaction protocols decide how multi-operation updates preserve atomicity and isolation.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.